Prezime i ime učenika : *Stoiljković Mihajlo*

Odeljenje i grupa : *IV4 – grupa 3.*

R. broj i naziv vežbe : *12.PIR сензор, детекција покрета, комбиновање са камером, прављење фотографија/видеа при детекцији покрета*

IZVEŠTAJ:

У овој вежби се врши пример једног видео надзора користећи технологије микрорачунарског система, PIR сензора као и програмирања у Python програмском језику.

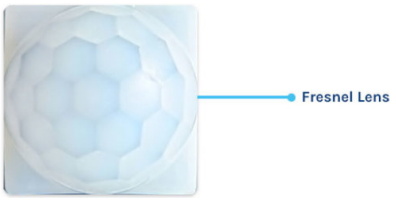
PIR сензор (Passive Infrared Sensor)



PIR сензор је сензор који детектује покрет тако што региструје промене у инфрацрвеном (IR) зрачењу које емитују људи и животиње. Назива се пасивни јер не емитује ништа, већ само прима инфрацрвено зрачење из околине. Често се користи у подручју сигурности (апликације, алармни системи и слично). Инфрацрвене зраке емитују тела предмета, људи или кућних љубимаца. Човек има око **37°C** → емитује IR зрачење таласне дужине ~8–14 μm. Баш у том опсегу PIR сензор је најосетљивији.

Најчешће коришћен модул јесте **HC-SR501** који је приказан на слици изнад. Сама унутрашња структура је **френел сочиво** односно бела купола чија улога је:

- Фокусирање IR зрачења
- Дељење простора на више зона (лево/десно)
- Повећава домет



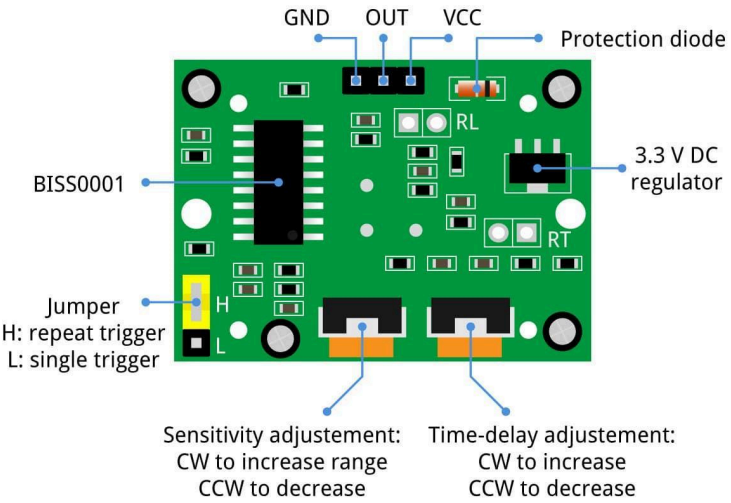
Без тог сочива, сензор би био много мање осетљив. Само срце система јесте пироелектрични кристал. **Пироелектрични ефекат значи да када се температура кристала промени, ствара се мали електрични напон**. Он реагује само на промену температуре а не на константну топлоту због чега се и често користи код сигурносних система (Ако човек стоји мирно, нема сигнала а уколуко се креће, појављује се сигнал).

Пироелектрични део има два дела:

- [+]
- [-]

Они су повезани тако да ако обе зоне виде исту температуру, нема сигнала а уколико има разлике, долази до детекције. Ово спречава лажне аларме од равномерне промене температуре. Често PIR модули користе чип **BISS0001** који врши појачање слабог сигнала, филтрирање шума и слилно. Сумирано овако функционише сам сензор. Френел сочиво дели просто на више зона, и када човек пролази, прво загреје једну зону па следећу и тако на даље и ствара серију

позитивних и негативних импулса. Излаз самог сензора је дигиталан и повезује се као дигитални сензор који има две могуће вредности (0 и 1). Логичка јединица има напон од 3.3v где сензор ради на тих 3.3v. Занимљивост јесте да сензор може да се прикључи и на 5v јер поседује линеарни регулатор напона који поставља напонски ниво на 3.3v па не може да дође до превисоког напона и оштећења сензора што је нама битно с обзиром да користимо Raspberry Pi микрорачунар чији је радни напон GPIO пинова 3.3v. Компоненте односно делови сензора на aPCB-у је приказан на слици испод.



PIR	**RPi**
VCC	5V
GND	GND
OUT	GPIO пин (нпр. 17)

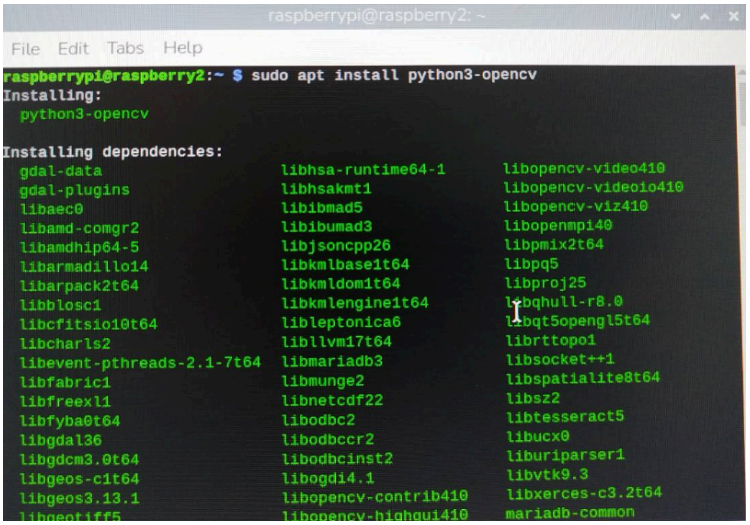
Инсталација потребних пакета

Пре него што почнемо са програмирањем микрорачунара односно писањем Python скрипте, потребно је да инсталирамо пакете за рад унутар Linux терминала. На почетку уносимо команду за ажурирање листе доступних пакета из репозиторијума.

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt update
```

Када унесемо ову команду у терминалу, систем контактира репозиторијуме (сервере), преузима најновије информације о пакетима и ажурира локалну базу података у софтверу. Након овога уносимо команду за инсталацију **openCV** Python библиотеке за рад са камером.

```
mihajlo@DESKTOP-BAFHUP9:~$ sudo apt install python3-opencv
```



Након инсталације библиотеке за рад са камером, уколико немамо инсталирамо Python библитеку за рад са GPIO пиновима.

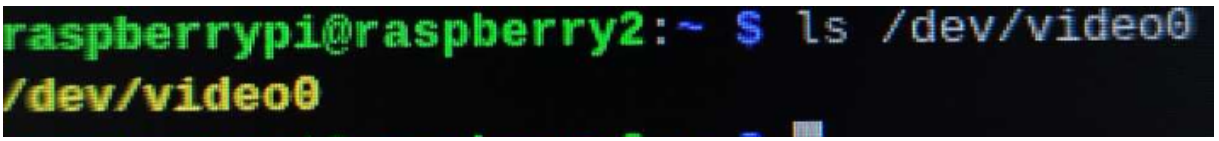
```
mihajlo@DESKTOP-BAFHUP9:~$ pip install RPi.GPIO
```

Након инсталације библиотека, потребно је прикључити нашу USB камеру на микрорачунар. Камера је произвољног типа, она служи за сликање и снимање када према програму задамо да изврши то. Након прикључувања камере потребно је да проверимо да ли камера ради преко листинг команде у терминалу.

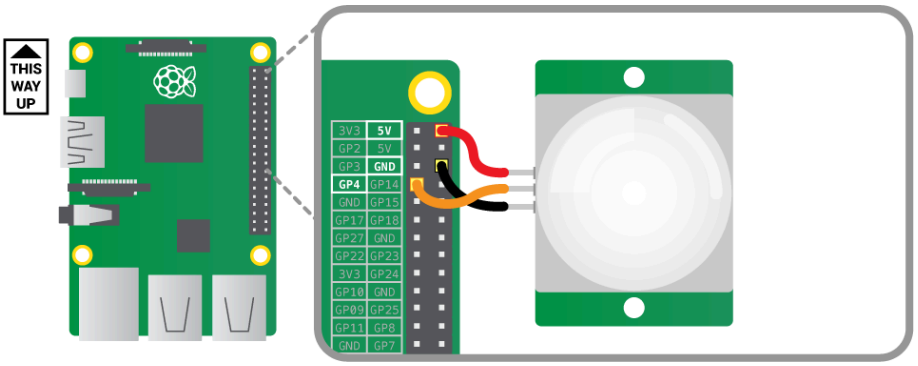


```
mihajlo@DESKTOP-BAFHUP9:~$ ls /dev/video0
```

Ако постоји `/dev/video0`, камера је препозната. То треба да изгледа овако:



Начин повезивања сензора са микрорачунаром треба да изгледа овако:



Тест скрипта

Након повезивања и провере, можемо кренути са писањен скрипти у Python-у. Прва скрипта ће бити тест скрипта у којој проверавамо рад PIR сензора, уколико ради све како треба онда можемо прећи на комплексији део.

```
import RPi.GPIO as GPIO
import time
PIR_PIN = 17 # digitalni pin PIR senzora
GPIO.setmode(GPIO.BCM) # numeracija procesora
GPIO.setup(PIR_PIN, GPIO.IN)
print("Test PIR senzora...")
try:
    while True:
        if GPIO.input(PIR_PIN):
            print("Pokret detektovan!")
        else:
            print("Nema pokreta")
        time.sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Објашњење скрипте:

На почетку увозимо библиотеку за рад са GPIO пиновима као и **time** модул за рад са временом. Затим дефинишемо променљиво која представља дигитални пин на коме је повезан PIR сензор. У следеће две линије вршимо подешавање пинова, као улазни и нумерација BCM за процесор. Вршимо обраду изузетака и ако се ухвати изузетак `KeyboardInterrupt,` вршимо подешавање GPIO пинова на стању високе импедансе. Унутар бесконачне петље, користимо input() функцију за проверу да ли имамо улаз са тог пина. Уколико има онда принтујемо да је покрет детектован, уколико није онда нема покрета и пауза од једне секунде у свакој итерацији петље. Ова скрипта ће се после надоградити имплементацијом библиотека за рад са камером.

Уколико сензор исписује податке у shell-u, можемо извршити наредни део вежбе и то реализација надзорног система тако што се детекцијом покрета врши усликавање уз помоћ камере.

Фотографија при детекцији покрета (USB камера + OpenCV Python)

У овом делу, пишемо скрипту у којој се реализује један мини надзорни систем у коме се врши фотографисање уз помоћ усб камере на основу детектованог покрета PIR сензора.

```
import RPi.GPIO as GPIO
import time
import cv2
from datetime import datetime

PIR_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)

camera = cv2.VideoCapture(0)
print("Sistem pokrenut...")

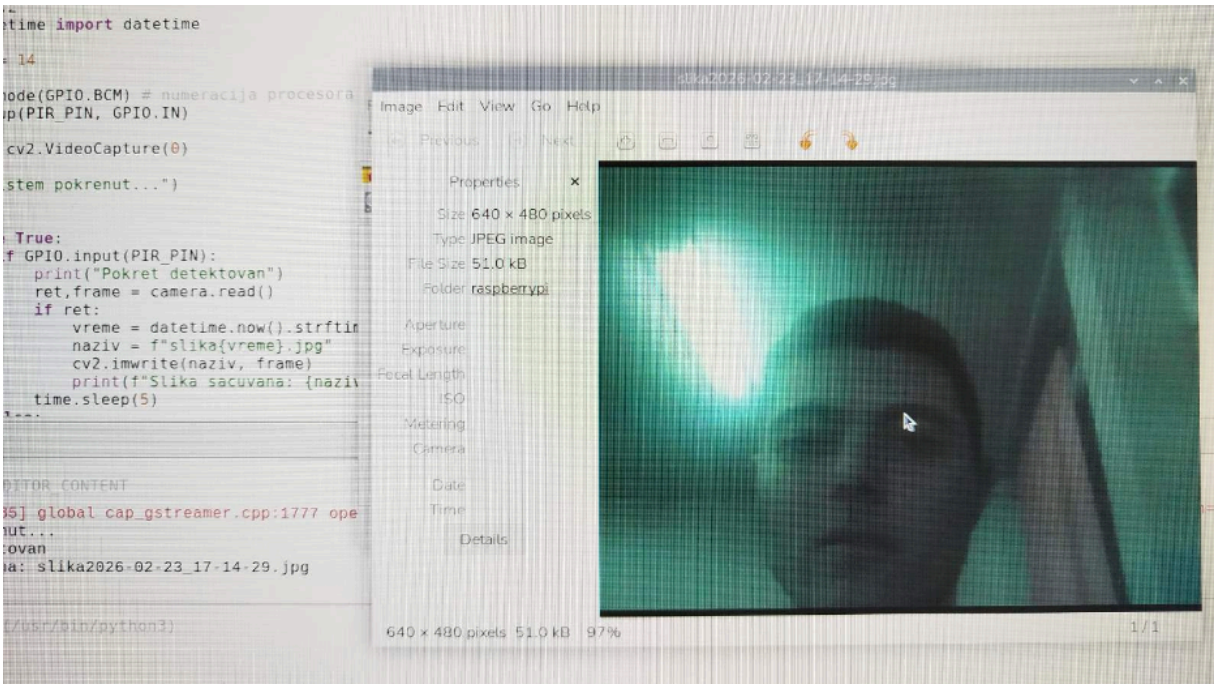
try:
    while True:
        if GPIO.input(PIR_PIN):
            print("Pokret detektovan!")
            ret, frame = camera.read()
            if ret:
                vreme = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
                naziv = f"slika_{vreme}.jpg"
                cv2.imwrite(naziv, frame)
                print(f"Slika sačuvana: {naziv}")
            time.sleep(5)
        time.sleep(0.5)
except KeyboardInterrupt:
    print("Prekid")
finally:
    camera.release()
    GPIO.cleanup()
```

Ова скрипта има структуру као и тест скрипта, али има додатне функционалности конкретно за камера систем. На почетку креирамо инстанцу класе `VideoCapture` која долази из C++ дела. Увози се модул **cv2**. Након тога исписујемо поруку да је систем покренут. Након тога вршимо обраду изузетака где уколико имамо неки улаз са пина на коме је PIR сензор, дефинишемо две промењиве где је:

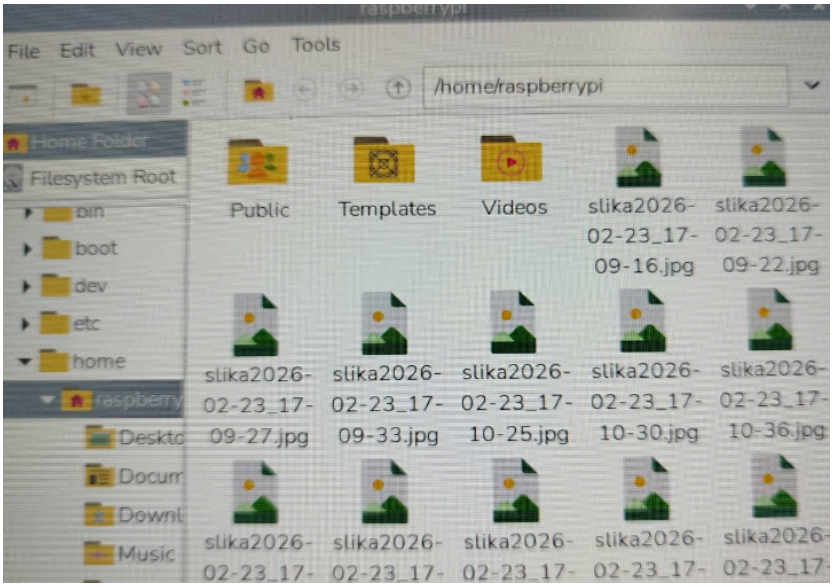
- ret - boolean: True ако је фрејм успешно прочитан, False ако није
- frame – NumPy array односно низ који садржи пикселе слике

Након тога дефинишемо промењиву време која служи за рачунање времена у тренутку фотографисања као и назив којом се дефинише назив слике. Након тога вршимо чување слике под задатим називом и фрејмом и исписујемо а је слика сачувана. Након тога правимо паузу од 5 секунди за читање улаза и пола секунду за итерацију петље. Остатак програма је исти, с тим што на крају у finally блоку вршимо отпуштање камере.

```
Sistem pokrenut...
Pokret detektovan
Slika sacuvana: slika2026-02-23_17-11-50.jpg
```



Приликом чувања слике, чува се у директојијуму на локацији **/home/raspberrypi**





Снимање видеа при детекцији покрета

У овој скрипти, врши се покретање снимања помоћу USB камере 10 секунди када детектује покрет.

```
import RPi.GPIO as GPIO
import time
import cv2
from datetime import datetime

PIR_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)

camera = cv2.VideoCapture(0)
fourcc = cv2.VideoWriter_fourcc(*'XVID')
print("Video nadzor aktivan...")

try:
    while True:
        if GPIO.input(PIR_PIN):
            print("Pokret detektovan - snimanje videa")
            vreme = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
            naziv = f"video_{vreme}.avi"
            out = cv2.VideoWriter(naziv, fourcc, 20.0, (640, 480))
            start_time = time.time()
            while int(time.time() - start_time) < 10:
                ret, frame = camera.read()
                if ret:
                    out.write(frame)
            out.release()
            print("Video sačuvan:", naziv)
            time.sleep(5)
        time.sleep(0.5)
except KeyboardInterrupt:
    print("Prekid")
finally:
    camera.release()
    GPIO.cleanup()
```

Објашњење скрипте:

Скрипта је слична као и за фотографисање, с тим што користимо додатне класе које омогућавају снимање. У почетку вршимо видео компресију где XVID представља назив кодека где * представља разлажење стринга у 4 аргумента. Тако да је ово еквивалентно:

```
fourcc = cv2.VideoWriter_fourcc('X','V','I','D')
```

Након тога користимо out промењиву која врши запис у 20'фрејма на задатој резолуцији. Док је време мање од 10 секунди камера врши читање, након тога се снимање завршава, чува фајл са задатим именом и остатак скрипте остаје исти.

```
Video nadzor aktivan...
Pokret detektovan - snimanje videa
Video sacuvan: video_2026-02-23_17-23-04.avi
Pokret detektovan - snimanje videa
```

